

计算思维与人工智能

实验五 结构体

GO!
TAKE YOU ON A TRIP

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

探索 从未停止

实验目的

- 理解结构体的存储本质，以及结构体与指针的关联关系
- 熟练运用指针操作结构体和结构体数组
- 掌握函数间传递结构体的方法，以及如何返回结构体变量的值

结构体的定义

在 C 语言中，**结构体（struct）** 是一种用户自定义的数据类型，可以将多个不同类型的数据“打包” 成一个整体，方便管理和使用。

结构体模板定义

```
struct student
{
    long studentID;
    char name[10];
    char sex;
    int  score[4];
};
```

```
typedef struct date
{
    int year;
    char month[10];
    int day;
}DATE;
```

结构体变量声明

```
struct student stu1;
struct student stu2,stu3;

typedef struct student STUDENT;
STUDENT stu1,stu2;(更简洁)
```

嵌套的结构体

```
struct student
{
    long studentID;
    char name[10];
    char sex;
    DATE birthday;
    int  score[4];
};
```

结构体所占内存的字节数

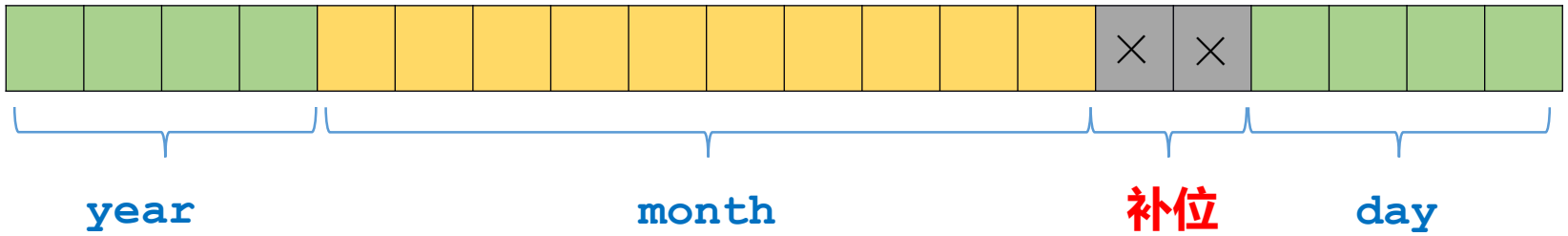
➢ 结构体类型的变量占用的内存字节数是多少？是所有成员总和吗？

CPU 访问内存不是逐个字节读，而是按固定大小的块读取。C语言为了提高 CPU 访问内存的效率，会对结构体成员进行**内存对齐**，规则通常是：

- 每个成员的起始地址必须是其自身大小（或对齐单位）的整数倍；
- 整个结构体的大小必须是最大成员对齐要求的整数倍。

```
typedef struct date
{
    int year;
    char month[10];
    int day;
}DATE;
printf("DATE: bytes = %d \n\n",
sizeof(DATE));
```

```
DATE: bytes = 20
```



结构体变量的初始化和引用

- 声明结构体变量时，直接用大括号整体初始化（按顺序）

```
STUDENT stu1 = { 2025001, "罗小黑", 'M', {2015, "May", 15}, {85, 92,
                 78, 90} };
```

- 成员访问：成员选择运算符（也称圆点运算符）

```
printf("学号: %ld\n", stu1.studentID);
printf("姓名: %s\n", stu1.name);
printf("性别: %c\n", stu1.sex);
printf("生日: %d年%s月%d日\n",
    stu1.birthday.year, //嵌套时以级联方式访问
    stu1.birthday.month,
    stu1.birthday.day);
printf("4门成绩: %d, %d, %d, %d\n",
    stu1.score[0], stu1.score[1],
    stu1.score[2], stu1.score[3]);
```

```
// 也可用'.'指定成员名初始化（顺序可乱）

STUDENT stu2 = {
    .studentID = 2025002,
    .sex = 'F',
    .birthday = {.year = 2014,
        .month = "July", .day = 22},
    .name = "鹿野",
    .score = {95, 88, 93, 89}
};
```

指针和结构体

- 定义指向结构体变量的指针

```
STUDENT stu1 = { 2025001, "罗小黑", 'M', {2015, "May", 15}, {85, 92,
                 78, 90} };
STUDENT *pt = &stu1;
```

- 使用指针访问结构体成员：指向运算符

```
(*pt).studentID = 2025003    //成员选择运算符
pt -> studentID = 2025003    //指向运算符（也称箭头运算符）
pt ->birthday.year = 1999    //级联访问
```

- 定义指向结构体数组的指针

```
STUDENT stu[2] = {{ 2025001, "罗小黑", 'M', {2015, "May", 15}, {85, 92, 78, 90}}
    ,{ 2025002, "鹿野", 'M', {2014, "July", 22}, {95, 88, 93, 89}};

STUDENT *pt = stu; or STUDENT *pt = &stu[0];
```

结构体的“浅拷贝”&“深拷贝”

- 浅拷贝： 如果结构体中有指针成员，那么只复制指针的值（即地址），不会复制指针指向的数据。

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char *name;
    int age;
} Person;

int main() {
    Person p1 ;
    p1.name = malloc(20 * sizeof(char));
    strcpy(p1.name, "Alice");
    p1.age = 30;

    Person p2 = p1;    // 浅拷贝

    printf("p1.name 存储的地址:%p,  指向的字符串为: %s\n", p1.name, p1.name);
    printf("p2.name 存储的地址:%p,  指向的字符串为 %s\n", p2.name, p2.name);

    strcpy(p2.name, "Bob");    // 修改 p2.name 指向的内容
    printf("\n***After change:***\n");
    printf("p1.name 存储的地址:%p,  指向的字符串: %s\n", p1.name, p1.name);
    printf("p2.name 存储的地址:%p,  指向的字符串: %s\n", p2.name, p2.name);

    free(p1.name);
    // free(p2.name); // 危险！重复释放同一块内存 → 未定义行为
    return 0;
}
```

```
D:\code\c\2025\test33\bin\Debug\test33.exe
p1.name 存储的地址:000001FB3347DF70,  指向的字符串为: Alice
p2.name 存储的地址:000001FB3347DF70,  指向的字符串为 Alice

***After change:***
p1.name 存储的地址:000001FB3347DF70,  指向的字符串: Bob
p2.name 存储的地址:000001FB3347DF70,  指向的字符串: Bob

Process returned 0 (0x0)   execution time : 0.107 s
Press any key to continue.
```

结果：两个结构体的指针指向同一块内存

结构体的“浅拷贝”&“深拷贝”

- 深拷贝： 不仅复制结构体本身，还为指针成员分配新的内存，并复制指向的数据

```
Person deep_copy_person(const Person *src) {
    Person dest;
    dest.age = src->age;
    dest.name = malloc(strlen(src->name) + 1); // 分配新内存
    strcpy(dest.name, src->name);            // 复制内容
    return dest;
}

int main() {
    Person p1;
    p1.name = malloc(20 * sizeof(char));
    strcpy(p1.name, "Alice");
    p1.age = 30;

    Person p2 = deep_copy_person(&p1);   // 深拷贝

    printf("p1.name 存储的地址:%p,  指向的字符串为: %s\n", p1.name, p1.name);
    printf("p2.name 存储的地址:%p,  指向的字符串为 %s\n", p2.name, p2.name);

    strcpy(p2.name, "Bob");
    printf("\n***After change:***\n");
    printf("p1.name 存储的地址:%p,  指向的字符串为: %s\n", p1.name, p1.name);
    printf("p2.name 存储的地址:%p,  指向的字符串为 %s\n", p2.name, p2.name);

    free(p1.name);
    free(p2.name); // 安全：两块独立内存
    return 0;
}
```

```
D:\code\c\2025\test33\bin\Debug\test33.exe
p1.name 存储的地址:000001FDA958E170,  指向的字符串为: Alice
p2.name 存储的地址:000001FDA9583380,  指向的字符串为 Alice

***After change:***
p1.name 存储的地址:000001FDA958E170,  指向的字符串为: Alice
p2.name 存储的地址:000001FDA9583380,  指向的字符串为 Bob

Process returned 0 (0x0)   execution time : 0.089 s
Press any key to continue.
```

结果：两个结构体完全独立，互不影响

向函数传递结构体

```
int main()
{
  struct date d;
  d.year = 1999;
  d.month = 4;
  d.day = 23;
  printf("Before function call:%d/%02d/%02d\n",
         d.year, d.month, d.day);
  Func(d);   /*结构体变量作函数实参，传值调用*/
  printf("After function call:%d/%02d/%02d\n",
         d.year, d.month, d.day);
  return 0;
}
```

值传递 → 无法改变外面的结构体

结构体变量作函数参数

```
Before function call:1999/04/23
After function call:1999/04/23
```

函数对结构体内容的修改不影响原结构体

```
struct date
{
   int year;
   int month;
   int day;
};
void Func(struct date p)
{
    p.year = 2000;
    p.month = 5;
    p.day = 22;
}
```

向函数传递结构体

```
int main()
{
  struct date d;
  d.year = 1999;
  d.month = 4;
  d.day = 23;
  printf("Before function call:%d/%02d/%02d\n",
         d.year, d.month, d.day);
  Func( &d ); /*结构体变量的地址作函数实参，传地址调用 */
  printf("After function call:%d/%02d/%02d\n",
         d.year, d.month, d.day);
  return 0;
}
```

地址传递 → 可以改变外面的结构体

结构体指针作函数参数

结构体指针作函数形参

实参必须为地址值

```
struct date
{
  int year;
  int month;
  int day;
};
void Func(struct date *p)
{
   p->year = 2000;
   p->month = 5;
   p->day = 22;
}
```

```
Before function call:1999/04/23
After function call:2000/05/22
```

函数返回结构体变量的值

```
int main()
{
  struct date d;
  d.year = 1999;
  d.month = 4;
  d.day = 23;
  printf("Before function call:%d/%02d/%02d\n",
         d.year, d.month, d.day);
  d = Func(d); /* 结构体变量作函数实参，传值调用 */
  printf("After function call:%d/%02d/%02d\n",
         d.year, d.month, d.day);
  return 0;
}
```

结构体变量
做函数返回值

```
struct date
{
   int year;
   int month;
   int day;
};
struct date Func(struct date p)
{
    p.year = 2000;
    p.month = 5;
    p.day = 22;
    return p;
}
```

返回结构体变量的值

也可以得到修改的结构体内容，但效率低

实验内容

5001 歌唱比赛（50%）

n(n≤100)名同学参加歌唱比赛，并接受 m(m≤20)名评委的评分，评分范围是 0到 10分。这名同学的得分就是这些评委给分中去掉一个最高分，去掉一个最低分，剩下 m-2 个评分的平均数。请问得分最高的同学分数是多少？评分保留 2 位小数。

【输入描述】

第一行两个整数 n, m。

接下来 n 行，每行各 m个整数，表示得分。

【输入描述】

输出分数最高的同学的分数，保留两位小数

【输入示例】

```
7 6
4 7 2 6 10 7
0 5 0 10 3 10
2 6 8 4 3 6
6 3 6 7 5 8
5 9 3 3 8 1
5 9 9 3 2 0
5 8 0 4 1 10
```

【输出示例】

```
6.00
```

实验内容

5002 新生舞会（50%）

新生舞会开始了。n名新生每人有三个属性：姓名、学号、性别。其中，

① 姓名用长度不超过20的仅由大小写字母构成的字符串表示；

② 学号用长度不超过10的 仅由数字构成的字符串表示；

③ 性别用一个大写字符 ‘F’ 或 ‘M’ 表示。

任意两人的姓名、学号均互不相同。换言之，每个人可被其姓名或学号唯一确定。给出m对两人的信息（姓名或学号），判断他们是否能共舞。两人能共舞的充要条件为两人性别相异。

实验内容

5002 新生舞会（50%）

【输入描述】

第一行一个整数n（2<=n<=1000），表示学生人数。接下来的n行每行依次包含一名新生的姓名、学号、性别，分别用一个空格隔开。

之后的一行是一个整数m(1<=m<=1000)，表示询问的数目。接着的m行每行包含两个信息（姓名或学号），保证两个信息不属于同一人，中间用一个空格隔开。

【输入描述】

对于每个询问输出一行，如果两人可以共舞，输出一个大写字母‘Y’，否则输出一个大写字母‘N’。

【输入示例】

```
4
John 10 M
Jack 11 M
Kate 20 F
Jim 21 M
3
John 11
20 Jack
Jim Jack
```

【输出示例】

```
N
Y
N
```

课程平台使用指南

- 登录网址：http://10.250.48.240:9000，推荐浏览器：Chrome；
- 初始用户名、密码均为学号，登录后请修改；
- 提交截止时间内，可以重新提交作业，不限次数；
- 只要有一次提交通过，即使重新提交代码未通过，也视为access状态；
- 目前仅支持C99，请在本地IDE完成调试后再提交课程平台；
- 代码没有点击“提交”之前，所有内容没有保存，请做好本地备份；
- 代码“提交”，需刷新之后，才可以查看“提交记录”和“已提交代码“；
- 只支持单文件；

课程平台使用指南

- 常见oj错误提示：

① Compile Error　编译错误，不符合语法规范

② Wrong Output　表示运行结果不正确

③ Representation error　表示输出格式不正确

④ Runtime error　请检查是否有除零、数组越界、eles语句块没有用{}括起来等

⑤ Nonzero Exit Status　一般表示主函数返回非0值



FEEL THE MEANING OF THE TRIP

请同学们开始实验

TAKE YOU ON A TRIP

GO!

哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

探索 从未停止